

Caller ID / Reverse Phone Lookup — Teardown

Target: `com.callerid.spamcallblocker.callapp.dialer.contacts` v4.0.3 (.xapk from APKPure) **Publisher brand:** "Auzi" / Auziapps (privacy policy at <https://www.auziapps.com/privacy-policy>) **Method:** apktool 3.0.2 decode + jadx 1.5.5 decompile. App code is R8-obfuscated (single-letter packages) but the app's own `com.callerid.*` tree and string/raw resources are readable; Retrofit HTTP annotations survived.

Summary

It is a full default-dialer / caller-ID app whose number→name resolution is a **server-side reverse-lookup against a crowd-sourced contacts database**, not a bundled offline DB and not a 3rd-party SDK (no Truecaller/Hiya/CallApp SDK present). At call time it POSTs the incoming phone number to a single AWS API Gateway → Lambda endpoint (`trucallercontacts_API`) with an `x-api-key` , and renders the returned name/email/country. Separately, it backs up the user's contacts to a per-user AWS S3 bucket via Cognito auth. The endpoint name and the in-app consent copy ("Your contacts will be uploaded to our secure cloud servers") strongly indicate the same uploaded contacts feed the lookup database — the classic Truecaller-style model.

Permissions & call-handling

Full permission set (from `apktool/AndroidManifest.xml`):

- **Contacts/call data:** `READ_CONTACTS` , `WRITE_CONTACTS` , `READ_CALL_LOG` , `WRITE_CALL_LOG`
- **Phone/telecom:** `READ_PHONE_STATE` , `READ_PRIVILEGED_PHONE_STATE` , `CALL_PHONE` , `CALL_PRIVILEGED` , `ANSWER_PHONE_CALLS` , `MANAGE_OWN_CALLS` , `MODIFY_PHONE_STATE` , `BIND_INCALL_SERVICE` , `BIND_TELECOM_CONNECTION_SERVICE`
- **Other:** `INTERNET` , `RECORD_AUDIO` , `SYSTEM_ALERT_WINDOW` (overlay caller card), `POST_NOTIFICATIONS` , `RECEIVE_BOOT_COMPLETED` , `FOREGROUND_SERVICE` , `WAKE_LOCK` , `READ_SYNC_SETTINGS` , the AdServices/Ad-ID permissions.
- Notably **no** `READ_PHONE_NUMBERS` declared and **no** `SEND_SMS` .

Call-handling components (app-owned): - `core.AuziCallService` — declared as `android.telecom.InCallService` (`IN_CALL_SERVICE_UI` intent). This is the **default-dialer / InCallService** path, i.e. the app must be set as the default phone/dialer app to function (there is a `SetAppAsDefaultCallerActivity`). It is *not* a `CallScreeningService` — it relies on the `InCallService` role, which is why it requests full dialer privileges rather than the lighter call-screening role. - `core.CallConnectionService` — `android.telecom.ConnectionService` . - `core.fcm.MyFirebaseMessagingService` — FCM push. - Call-time resolution entry points: `AuziCallService.onCallAdded()` (log line "onCallAdded: Starting contact lookup for number: ") and `core.call.CallingActivity.lookupCallerName()` .

Caller-ID data source (the mechanism, with evidence)

Verdict: server-side reverse-lookup API backed by a crowd-sourced contacts DB. Evidence:

1. **One Retrofit endpoint, for number lookup.** `e6/a.java` (the only `@retrofit2.http` interface in the app): `java public interface a { @o("trucallercontacts_API") Object a(@og.a JsonObject body, @j Map<String,String> headers, d<? super AuziLookupNumberApisResponse> dVar); }`
`@o = @POST, @og.a = @Body, @j = @HeaderMap`. The endpoint path is literally `trucallercontacts_API` (Truecaller-style naming).
2. **What is sent / received.** `c6/w.java` builds the request body and headers: - Body: `JsonObject` with a single "phone" property. - Header: `map.put("x-api-key", MainActivity.f6659r)` where the key comes from string resource `headerKey` (obfuscated with a `"pAk9"+key+"90nIt"` wrapper before storage). - Response model `data/api/AuziLookupNumberResponseData.java` (gson `@SerializedName`): `code, uuid, email, name, country, phone, responseStatus`. So the server returns the **owner's name + email + country** for a number — consistent with a DB built from other users' address books, not a carrier/telco lookup.
3. **No bundled offline DB.** `assets/` contains only `countries.json` (country dial codes/flags), `builddatas.json`, and ad-SDK files — **no *.db / *.sqlite / prepopulated number DB**. The Room `AppDatabase` is local app state (contacts cache, blocklist, call-log models), not a caller-ID corpus.
4. **No 3rd-party caller-ID SDK.** No Truecaller SDK, Hiya, CallApp, NumVerify, Twilio Lookup, or Veriphone packages exist (the only "truecaller" string hits are substrings inside GMS internals). The lookup is entirely the publisher's own AWS backend.
5. **Crowd-source upload (strongly implied).** The in-app consent string (`res/values/strings.xml`):

"Your contacts will be uploaded to our secure cloud servers to backup and restore. By tapping Allow Access you consent to upload and securely backup your contacts."

Combined with the `trucallercontacts_API` endpoint name and a name/email-returning response, the uploaded address books are the lookup corpus. Caveat: the exact upload-to-lookup-DB write path is in obfuscated coroutine classes and I could not line-trace it end to end; the S3 backup (below) and the lookup API are architecturally separate channels, so the contacts→corpus ingestion likely happens server-side from the S3/contacts upload rather than via a second client call.

Call-time resolution path: incoming call → `AuziCallService.onCallAdded` (or `CallingActivity` on the call screen) → coroutine `lookupCallerName()` → `loginRetrofit2Api(number)` → POST to `trucallercontacts_API` → `AuziLookupNumberApisResponse.name` rendered on the overlay/call screen. (`CallingActivity.java` lines 84/166/375/385/1078/1768.)

Backend / endpoints

- **Base URL:** `https://yo7gqt6eu3.execute-api.us-east-1.amazonaws.com/default/` (string resource `baseurl`). AWS API Gateway (`us-east-1`) fronting a Lambda named in path `trucallercontacts_API`. Auth = `x-api-key` header (a static API-Gateway key shipped in the APK as `headerKey`).
- **Contact backup/restore = AWS Amplify + Cognito + S3** (`res/raw/amplifyconfiguration.json`, `awsconfiguration.json`):
- Cognito User Pool `us-east-1_gztcLm0yQ`, App Client `6l0no0ag7chf0jjmo4c74592o1`, signup attr = EMAIL (email/password, SRP auth, MFA off).

- Cognito Identity Pool `us-east-1:c033d493-717d-4641-8b24-a997a91f38ce`.
- S3 bucket `contactbackupcybrond8700-dev` (`us-east-1`), default access level `guest`, via `com.amazonaws.mobileconnectors.s3.transferutility.TransferService` (declared in manifest). Note the `-dev` suffix — a dev-stage bucket shipped in production.
- **Firestore project** `caller-id---phone-lookup` (`google_app_id 1:247678257496:android:... , sender 247678257496`, storage bucket `caller-id---phone-lookup.firebaseio.com`). Used for FCM push + Firebase phone-auth (`securetoken.google.com/caller-id---phone-lookup /verifyPhoneNumber`). So there are effectively **two backends**: API-Gateway/Lambda (lookup) + Amplify/Cognito/S3 (contact backup), plus Firebase for messaging/phone-verify.

3rd-party SDKs

- **Ads (mediation)**: Google AdMob (GMS Ads) is primary. AdMob App ID `ca-app-pub-2227737204422905~9079147810`; ~9 live ad-unit IDs under publisher `2227737204422905` (interstitial/native/rewarded/banner — confirmed by string flags like `LookupResultsRewarded0n0ff`, `LookupNative0n0ff`, `LookupBackupInter0n0ff`). Mediation adapters bundled: **AppLovin**, **Meta/Facebook Audience Network** (`assets/audience_network.dex`), **Mintegral/MBridge**, **InMobi**.
- **Analytics/infra**: Firebase (Analytics/Masurement, Crashlytics, Sessions, FCM), AWS Amplify/Cognito/S3, Retrofit2 + OkHttp3 + Gson, Room, WorkManager, Glide/Picasso, ezvcard (vCard handling), Google Play Billing.

Monetization

- **Subscriptions + lifetime IAP** (Play Billing, `subs`). SKU-ish ids in app: `monthly_pre`, `yearly_pre`, `monthsub_30off`, `monthsub_50off`, `yearsub_30off`, `yearsub_50off`, `lifetime_plan`, `contactbackup_premium`. Paywall gates: free tier = limited lookups (`FREE_LOOKUP_LIMIT`, "One Lookup Remaining", watch-ad-to-unlock-1-lookup) and **50 free contact backups**, then "Go Premium for unlimited lookups / unlimited backups."
- **Ads** as above (rewarded ad to unlock an extra lookup; interstitials around lookup/backup/restore flows).

Feasibility for reusing sign-up phone numbers (Cloud Vault)

Technical mechanism, restated plainly: a caller-ID feature is just a `phone → {name}` key-value store plus a real-time client lookup. This app builds that store by **harvesting users' on-device address books** (each upload contributes "this number is labelled X in someone's phonebook"), aggregating server-side, and serving it back. A single phone number captured at *your* sign-up is only useful as one row; it is the **graph of everyone's contacts** that produces the name for an unknown caller. So a cloud-backup app could, in principle, accumulate the same corpus *if and only if* it (a) reads users' `READ_CONTACTS`, (b) uploads those contact name↔number pairs to a server, and (c) exposes a lookup API. Cloud Vault already captures phone numbers and (potentially) contacts at backup — the data would be technically reusable to seed such a store.

But the hard blockers make this a bad idea for Cloud Vault:

1. **Google Play "Don't repurpose Contacts/Call-Log data" policy.** Play's User Data + Permissions policy requires personal/sensitive data (Contacts especially) be used **only for the purpose the user consented to and that's disclosed in the listing**. Backing up a user's own contacts is a permitted

purpose; **silently feeding those contacts into a third-party caller-ID lookup that exposes their friends' names to strangers is a different, undisclosed purpose** and a classic policy-rejection / app-suspension trigger. This is exactly the use the teardown app papers over with an explicit consent screen — and even with consent, repurposing for a *separate* product is not clearly allowed.

2. **Call Log & SMS permission policy.** `READ_CALL_LOG` / `WRITE_CALL_LOG` are restricted permissions only grantable to apps whose **core, default-handler functionality** requires them (default Dialer/Phone, default Assistant, etc.). A cloud-backup app does not qualify and would be **removed from Play for requesting them**. Caller-ID/spam features that touch the call screen require being the **default dialer (InCallService)** — as this app is — or a `CallScreeningService`. A backup app cannot bolt that on without becoming a dialer and passing the restricted-permission Declaration Form + a video review.
3. **Privacy / consent of the third parties.** The names being served belong to people who never installed either app (they're entries in someone else's phonebook). This is the core legal exposure of the Truecaller model — GDPR/CCPA "no lawful basis" complaints, and it is precisely why these apps run dedicated brands and dev-stage AWS, not a respectable consumer-backup brand. Attaching it to Cloud Vault imports that liability onto the partnership's main product.

Verdict for the partnership: the mechanism is cheap to replicate technically (one Lambda + a `phone→name` table fed by contact uploads), and there is no proprietary SDK or licensed data feed to license — it's all self-hosted AWS. But it is **not safely combinable with a cloud-backup app**: it would require Cloud Vault to either repurpose already-consented backup data for an undisclosed caller-ID product (Play policy violation), or become a full default-dialer to do real call-time screening (restricted-permission gate + review). If the publisher wants caller-ID, it belongs in a **separate, single-purpose dialer/caller-ID app** with its own listing, consent flow, and restricted-permission declaration — which is exactly how this target ships it. Reusing Cloud Vault's sign-up phone numbers as the seed corpus does not meaningfully shortcut that; the corpus value is in scraped contact *graphs*, not the signup number itself.